

Lab Work 10

Topic	Using AI to generate docstrings, comments, explanations, and potentially diagram descriptions.
Objective	Utilize an AI assistant to efficiently document the OOP project.
AI Copilot Usage	Allowed and encouraged

Important Notes

- This is an individual task. All code must be your own original work.
- Ask questions and seek help from the lecturer or teaching assistants if you are stuck.
- Focus on writing clean, well-organized, and well-documented code.

Submission

- Submit updated ***school_simulator.py*** (with improved documentation), any separate explanation/diagram files, and your prompt history for this session.

Tasks

1. Generating Comprehensive Docstrings:

- Target:** Choose a class like ***Unit*** or ***EnrollmentSystem***.
- Go through its methods.
- Prompt:** For a method, copy its signature and code. Paste into your AI assistant and ask: "Generate a detailed Python docstring for this method using Google style format. Include descriptions for parameters (Args), return values (Returns), and any custom exceptions it might raise (Raises)."
- Evaluate:** Review the generated docstring. Is it accurate? Does it list parameters, return types, and exceptions correctly? Integrate it into your code, refining as needed. Repeat for other key methods and the class docstring itself.

2. Generating Explanatory Code Comments:

- Target:** Find a potentially complex code block (e.g., inside ***PersonFactory.create_person*** or ***EnrollmentSystem.enroll_student_in_un*** ***it***).
- Prompt:** Copy the code block. Paste into your AI assistant and ask: "Add explanatory code comments to this Python code block to clarify the logic, especially the purpose of different checks or error handling."

- c) **Evaluate:** Read the comments added by the AI. Are they clarifying the *why*? Remove comments that just restate the obvious code. Integrate useful comments.
3. **Generating Explanations and Descriptions:**
- a) **Prompt (Overall System):** Ask your AI assistant: "Write a short summary describing the purpose and relationships between the main classes (Person, Student, Teacher, Unit, EnrollmentSystem, SchoolRegistry) in my Python school simulator project." (You might need to provide class names or snippets if the AI lacks context).
 - b) **Prompt (Specific Interaction):** "Explain, in plain English, the steps involved when a student enrolls in a unit using the **EnrollmentSystem**, mentioning the roles of the Registry and the Unit class."
 - c) **Evaluate:** Check the generated text for clarity and accuracy. Save this text - it could be useful for a README.
4. **Generating Diagram Descriptions (Textual/Simple Syntax):**
- a) **Prompt:** Ask your AI assistant: "Generate PlantUML (or Mermaid) syntax for a simple class diagram. It should show that Student and Teacher inherit from an abstract Person class. It should also show an association relationship between Student and Unit."
 - b) **Evaluate:** Look at the generated text syntax. Does it correctly represent the requested relationships (<|-- for inheritance, -- or o-- for association)? Copy the syntax to save it. (Visualization is optional).
5. **Creative Challenge (Optional, if time permits):**
- a) **Generate a Draft README:** Ask the AI assistant: "Generate a draft README.md file for my Python **school_simulator.py** project. Include sections for: Project Title, Description, Features, Getting Started (how to run), Main Classes, and How to Run Tests (mentioning **unittest** or **pytest**). Use the explanations generated earlier if possible." Refine the generated Markdown.